

COL100: Introduction to Computer Science

Lab 11: Class and Object

March 31, 2025

1 Introduction

Classes are a fundamental concept in object-oriented programming (OOP), which organizes programs around data abstractions. A class is a blueprint for creating objects that encapsulate both data/attributes and methods to operate on that data. It allows users to define their own types through classes. These user-defined types enable more modular, reusable, and maintainable code, just as built-in types like `float` and `str` provide essential functionality in Python programs.

1.1 Class

A **class** is a template used to create objects. It defines attributes and methods that objects created from the class will have. For example, we can define a `Car` class to represent cars with attributes like `make`, `model`, and `year`, and methods like `display_info` to display information about the car.

Example of a simple class definition:

```
1 class Car:
2     def __init__(self, make, model, year):
3         self.make = make
4         self.model = model
5         self.year = year
6
7     def display_info(self):
8         print(f"{self.year} {self.make} {self.model}")
```

In this class:

- The `__init__` method initializes an object with specific values for its attributes.
- The `display_info` method prints information about the car.

1.2 Object

An **object** is an instance of a class. Once a class is defined, objects can be created using that class. For example, creating an object of the `Car` class:

```
1 my_car = Car("Toyota", "Corolla", 2021)
2 my_car.display_info()
```

This will output:

Here, `my_car` is an object of the `Car` class, and we can access its methods and attributes. Python allows programmers to define new types using classes that are as intuitive as built-in types. Magic methods, also known as dunder methods (double underscore methods), enable custom implementations of built-in functions, operators, and other behaviors. This document provides an overview of key magic methods used for infix operators.

1.3 Magic Methods

Magic methods are used to define operator behavior for custom classes. These methods make custom classes as intuitive and usable as Python's built-in types. Magic methods can be implemented to customize the behavior of built-in functions such as `str` and `len`.

Table 1: Magic Methods for Operators

Operator	Magic Method	Description
+	<code>__add__(self, other)</code>	Defines addition behavior.
-	<code>__sub__(self, other)</code>	Defines subtraction behavior.
*	<code>__mul__(self, other)</code>	Defines multiplication behavior.
/	<code>__truediv__(self, other)</code>	Defines true division behavior.
%	<code>__mod__(self, other)</code>	Defines modulus behavior.
**	<code>__pow__(self, other)</code>	Defines exponentiation behavior.
//	<code>__floordiv__(self, other)</code>	Defines floor division behavior.
>>	<code>__rshift__(self, other)</code>	Defines right shift behavior.
<<	<code>__lshift__(self, other)</code>	Defines left shift behavior.
&	<code>__and__(self, other)</code>	Defines bitwise AND behavior.
	<code>__or__(self, other)</code>	Defines bitwise OR behavior.
^	<code>__xor__(self, other)</code>	Defines bitwise XOR behavior.
==	<code>__eq__(self, other)</code>	Defines equality comparison behavior.
!=	<code>__ne__(self, other)</code>	Defines inequality comparison behavior.
<	<code>__lt__(self, other)</code>	Defines "less than" comparison.
≤	<code>__le__(self, other)</code>	Defines "less than or equal to" comparison.
>	<code>__gt__(self, other)</code>	Defines "greater than" comparison.
≥	<code>__ge__(self, other)</code>	Defines "greater than or equal to" comparison.

Example

Below is the implementation of a `Toy` class in Python, which demonstrates the use of magic methods to provide class-specific behavior.

```

1 class Toy(object):
2     def __init__(self):

```

```

3     self._elems = []
4
5     def add(self, new_elems):
6         """new_elems is a list"""
7         self._elems += new_elems
8
9     def __len__(self):
10        return len(self._elems)
11
12    def __add__(self, other):
13        new_toy = Toy()
14        new_toy._elems = self._elems + other._elems
15        return new_toy
16
17    def __eq__(self, other):
18        return self._elems == other._elems
19
20    def __str__(self):
21        return str(self._elems)
22
23    def __hash__(self):
24        return id(self)
25
26
27 t1 = Toy()
28 t2 = Toy()
29 t1.add([1, 2])
30 t2.add([3, 4])
31 t3 = t1 + t2
32 print('The value of t3 is', t3)
33 print('The length of t3 is', len(t3))
34 d = {t1: 'A', t2: 'B'}
35 print('The value', d[t1], 'is associated with the key t1 in d.')

```

Explanation

- The Toy class demonstrates the use of several magic methods:
 - `__init__`: Initializes the object with an empty list `_elems`.
 - `__len__`: Returns the length of `_elems`, enabling the use of `len()`.
 - `__add__`: Defines the `+` operator for combining two Toy objects.
 - `__eq__`: Compares two Toy objects for equality based on their elements.
 - `__str__`: Provides a string representation for the Toy object.
 - `__hash__`: Defines a hash function, allowing the object to be used as a dictionary key.
- Example operations:

- Adding elements to `t1` and `t2` using `add()`.
- Combining `t1` and `t2` to create `t3` using the `+` operator.
- Accessing the length of `t3` using `len()`.
- Using `Toy` objects as keys in a dictionary.

Guidelines for Implementation

While Python allows flexibility in defining these operators, it is strongly recommended to adhere to their conventional meanings to maintain code clarity and avoid confusion. Table 1.3 shows the magic methods with respect to the operators with description.

2 Submission Problem

2.1 Display Features of a Car

2.1.1 Problem Statement

Earlier in this document, we created a `Car` class and defined its attributes and methods. Now the task is to extend on it. Create a `Car` class that has the following attributes:

- `make` (string)
- `model` (string)
- `year` (integer)
- `mileage` (integer)
- `task_size` (integer)

The class should have the following methods:

- `display_info()`: Prints year, make, model and mileage.
- `get_mileage()`: Prints the current mileage of the car in km/l.
- `calculate_mileage(kms)`: Given a new set of distance travelled, calculate mileage, (assuming full tank). Also, update the value stored in the object. Do floor division while calculating mileage.

Sample Input

```
1 3
2 Toyota Camry 2020 10 10 # make model year mileage tank_capacity
3 calculate_mileage 150
4 get_mileage
5 display_info
```

Output

```
1 15
2 2020 Toyota Camry 15
```

Explanation You should define the class appropriately considering the given main program so that the given output can be printed.

3 Practice Problems

3.1 Calculate Area and Perimeter of a Rectangle

3.1.1 Problem Statement

Create a `Rectangle` class with the following attributes:

- `length` (float)
- `width` (float)

The class should have the following methods:

- `get_area()`: Returns the area of the rectangle (`length * width`).
- `get_perimeter()`: Returns the perimeter of the rectangle (`2 * (length + width)`).

3.1.2 Test Case 1

Main Program

```
1 rect = Rectangle(5, 10)
2 rect.get_area()
3 rect.get_perimeter()
```

Output

```
1 50
2 30
```

Explanation

In this test case, the input is a rectangle with length '5' and width '10'. The area is calculated as:

$$\text{Area} = 5 \times 10 = 50$$

The perimeter is calculated as:

$$\text{Perimeter} = 2 \times (5 + 10) = 30$$

Thus, the expected output is the area followed by the perimeter.

3.1.3 Test Case 2

Main Program

```
1 rect = Rectangle(7, 3)
2 rect.get_area()
3 rect.get_perimeter()
```

Output

```
1 21
2 20
```

Explanation

In this test case, the input is a rectangle with length '7' and width '3'. The area is calculated as:

$$\text{Area} = 7 \times 3 = 21$$

The perimeter is calculated as:

$$\text{Perimeter} = 2 \times (7 + 3) = 20$$

Thus, the expected output is the area followed by the perimeter.

3.2 Bank Account Operations

3.2.1 Problem Statement

Create a `BankAccount` class with the following attributes:

- `account_holder` (string)
- `balance` (float)

The class should have the following methods:

- `deposit(amount)`: Deposits the given amount into the account.
- `withdraw(amount)`: Withdraws the given amount from the account if there is enough balance.
- `get_balance()`: Returns the current balance of the account.

3.2.2 Test Case 1

Main Program

```
1 account = BankAccount("Alice", 500.0)
2 account.deposit(200.0)
3 account.withdraw(150.0)
4 account.get_balance()
```

Output

```
1 550.0
```

Explanation

- The program creates a 'BankAccount' object with:

- Account holder: “Alice”
- Initial balance: 500.0
- A deposit of 200.0 is made, updating the balance to ‘500.0 + 200.0 = 700.0’.
- A withdrawal of 150.0 is made, updating the balance to ‘700.0 - 150.0 = 550.0’.
- Thus, the expected output is the current balance: ‘550.0’.

3.2.3 Test Case 2

Main Program

```

1 account = BankAccount("Bob", 500.0)
2 account.deposit(200.0)
3 account.withdraw(800.0) # Attempting to withdraw more than the
   balance

```

Output

```

1 Insufficient funds

```

Explanation

- The input creates a ‘BankAccount’ object with:
 - Account holder: “Bob”
 - Initial balance: 500.0
- A deposit of 200.0 is made, updating the balance to: $500.0 + 200.0 = 700.0$
- A withdrawal of 800.0 is attempted, but since the balance is only 700.0, the withdrawal fails. The program outputs: “Insufficient funds”.

3.3 Area and Circumference of a Circle

3.3.1 Problem Statement

Create a `Circle` class with the following attributes:

- `radius` (float)

The class should have the following methods:

- `get_area()`: Returns the area of the circle ($\pi \times \text{radius}^2$).
- `get_circumference()`: Returns the circumference of the circle ($2 \times \pi \times \text{radius}$).
- A class method `get_pi()` that returns the value of π .

Then, write the program which takes the radius as input, and prints the area, circumference, and the value of π .

3.3.2 Test Case 1

Input

```
1 10
```

Output

```
1 314.1592653589793
2 62.83185307179586
3 3.141592653589793
```

Explanation

For a circle with a radius of 10, the area is calculated as $\pi \times 10^2 = 314.1592653589793$ and the circumference is calculated as $2 \times \pi \times 10 = 62.83185307179586$. The value of π is calculated as 3.141592653589793 by dividing the circumference by the diameter $2 \times 10 = 20$.

3.3.3 Test Case 2

Input

```
1 5
```

Output

```
1 78.53981633974483
2 31.41592653589793
3 3.141592653589793
```

Explanation

For a circle with a radius of 5, the area is calculated as $\pi \times 5^2 = 78.53981633974483$ and the circumference is calculated as $2 \times \pi \times 5 = 31.41592653589793$. The value of π is returned as 3.141592653589793 by dividing the circumference by the diameter $2 \times 5 = 10$.

3.4 Complex Calculator

3.4.1 Problem Statement

Design a class `Calculator` that performs basic arithmetic operations and supports additional operations like exponentiation and square root. The class should encapsulate the following private attributes:

- `_last_result`: Stores the last result computed.

The class should include the following methods:

- `add(a, b)`: Returns the sum of `a` and `b`.
- `subtract(a, b)`: Returns the result of `a` minus `b`.

- `multiply(a, b)`: Returns the result of `a` multiplied by `b`.
- `divide(a, b)`: Returns the result of `a` divided by `b` (throws an exception if `b == 0`).
- `exponentiate(a, b)`: Returns `a` raised to the power of `b`.
- `squareroot(a)`: Returns the square root of `a`.
- `get_last_result()`: Returns the last computed result.
- `clear()`: Resets the `_last_result` to 0.

Additionally, implement polymorphism to handle operations on both integers and floating-point numbers.

3.4.2 Test Case 1

Main Program

```

1 calc = Calculator()
2 calc.add(10, 5)
3 calc.subtract(10, 3)
4 calc.multiply(2, 3)
5 calc.divide(10, 0)
6 calc.exponentiate(2, 3)
7 print(calc.get_last_result())

```

Output

```

1 Result: 8

```

Explanation

Operations like addition, subtraction, multiplication, and exponentiation update the result stored in `_last_result`, which can be accessed via the `get_last_result()` method. Handling division by zero should raise an exception.

3.5 Event Calendar

3.5.1 Problem Statement

Create a class `EventCalendar` that manages events scheduled on different dates. The class should have the following private attributes:

- `_events`: A dictionary where keys are dates (in the format `YYYY-MM-DD`) and values are lists of event descriptions.

The class should include the following methods:

- `add_event(date, event)`: Adds an event to the specified date.
- `remove_event(date, event)`: Removes a specific event from a specified date.
- `get_events(date)`: Returns a list of all events scheduled on a given date.
- `get_all_events()`: Returns a dictionary of all events scheduled across all dates.
- `clear()`: Clears all events.
- `__str__()`: Returns a string representation of the calendar.

Ensure that no duplicate events are added for the same date.

3.5.2 Test Case 1

Main Program

```

1 calendar = EventCalendar()
2 calendar.add_event("2024-12-01", "Meeting with John")
3 calendar.add_event("2024-12-01", "Conference Call")
4 calendar.add_event("2024-12-02", "Team Dinner")
5 calendar.remove_event("2024-12-01", "Conference Call")
6 print(calendar.get_events("2024-12-01"))

```

Output

```

1 ['Meeting with John']

```

Explanation

The event “Conference Call” was removed from the list of events scheduled for “2024-12-01”. The remaining event is “Meeting with John”.

3.6 Chess Game

3.6.1 Problem Statement

Create a class-based implementation for a simple chess game.

- You need to represent the chessboard, pieces, and moves.
- The chessboard should be represented as an 8x8 grid, and each piece should be represented as a subclass of a base class `Piece`.
- The base class should have common methods for movement and capturing, while each piece subclass (e.g., `King`, `Queen`, `Knight`) should define its unique movement logic.

The following methods should be implemented:

- `move(from_position, to_position)`: Moves a piece from one position to another if the move is valid.
- `capture(piece)`: Captures another piece.
- `get_valid_moves(position)`: Returns a list of all valid moves for a piece from the given position.
- `is_checkmate()` or `is_stalemate()`: Determines if the current player is in checkmate or stalemate.

3.6.2 Test Case 1

Main Program

```

1 board = ChessBoard()
2 king = King()
3 queen = Queen()
4 board.place_piece(king, (0, 4)) # Place king at position (0,4)
5 board.place_piece(queen, (4, 4)) # Place queen at position (4,4)
6 king.move((0, 4), (1, 4)) # King moves
7 queen.capture(king)
8 print(board.is_checkmate())

```

Output

```

1 True

```

Explanation

The King was captured by the Queen, resulting in a checkmate condition, which is detected by the `is_checkmate()` method.

3.7 Car Rental Management System

3.7.1 Problem Statement

Design a class `CarRental` that manages a fleet of cars available for rent. Each car is represented by a `Car` class, and customers are represented by a `Customer` class. The system should support the following features:

- **Attributes:**
 - **CarRental Class:**
 - * `_cars`: A list of cars in the fleet.
 - * `_rented_cars`: A dictionary mapping customers to their rented cars.
 - **Car Class:**

- * `car_id`: A unique identifier for the car.
 - * `brand`: Brand of the car.
 - * `model`: Model of the car.
 - * `rental_price`: Price per day for renting the car.
 - * `available`: A boolean indicating if the car is available for rent.
- Customer Class:
 - * `name`: Name of the customer.
 - * `customer_id`: A unique identifier for the customer.
 - * `rented_cars`: A list of cars rented by the customer.
- Methods:
 - CarRental Class:
 - * `add_car(car)`: Adds a new car to the fleet.
 - * `remove_car(car_id)`: Removes a car from the fleet.
 - * `rent_car(customer, car_id)`: Allows a customer to rent a car.
 - * `return_car(customer, car_id)`: Allows a customer to return a rented car.
 - * `list_available_cars()`: Lists all available cars for rent.
 - * `list_rented_cars()`: Lists all currently rented cars and their customers.
 - Customer Class:
 - * `rent_car(car)`: Adds a car to the customer's rented cars.
 - * `return_car(car)`: Removes a car from the customer's rented cars.
 - Inheritance: Create subclasses of `Car` like `SUV`, `Sedan`, and `Truck`, each with unique attributes (e.g., `cargo_capacity` for `Truck`, `luxury_rating` for `Sedan`).

3.7.2 Test Case 1

Input:

```

1 # Create CarRental system
2 rental = CarRental()
3
4 # Add cars to the fleet
5 car1 = SUV("SUV001", "Toyota", "RAV4", 70, 500)
6 car2 = Sedan("SED001", "Mercedes", "C-Class", 120, 5)
7 car3 = Truck("TRK001", "Ford", "F-150", 90, 1000)
8
9 rental.add_car(car1)

```

```
10 rental.add_car(car2)
11 rental.add_car(car3)
12
13 # Create a customer
14 customer = Customer("John Doe", "C001")
15
16 # Rent a car
17 rental.rent_car(customer, "SUV001")
18
19 # List rented cars
20 print(rental.list_rented_cars())
21
22 # Return the car
23 rental.return_car(customer, "SUV001")
24
25 # List available cars
26 print(rental.list_available_cars())
```

Output:

```
1 Rented Cars:
2 John Doe: [Toyota RAV4]
3
4 Available Cars:
5 [ 'Toyota RAV4', 'Mercedes C-Class ', 'Ford F-150 ']
```